

SWIGGY SALES ANALYSIS

```
import os
print(os.listdir())
```

['.config', 'sample_data']

```
from google.colab import files
uploaded = files.upload()
```

Choose Files

swiggy_data.xlsx

swiggy_data.xlsx(application/vnd.openxmlformats-officedocument.spreadsheetml.sheet) - 10399966 bytes, last modified: 2/23/2026 - 100% done

Saving swiggy_data.xlsx to swiggy_data.xlsx

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

df = pd.read_excel("swiggy_data.xlsx")
```

```
import pandas as pd
df = pd.read_excel("swiggy_data.xlsx")
```

```
df.head()
```

	State	City	Order Date	Restaurant Name	Location	Category	Dish Name	Price (INR)	Rating	Rating Count
0	Karnataka	Bengaluru	2025-06-29	Anand Sweets & Savouries	Rajarajeshwari Nagar	Snack	Butter Murukku-200gm	133.9	4.0	0
1	Karnataka	Bengaluru	2025-04-03	Srinidhi Sagar Deluxe	Kengeri	Recommended	Badam Milk	52.0	4.5	25
2	Karnataka	Bengaluru	2025-01-15	Srinidhi Sagar Deluxe	Kengeri	Recommended	Chow Chow Bath	117.0	4.7	48

```
df.shape[0]
```

197430

```
print("no. of fields:", df.shape[1])
```

no. of fields: 10

```
df.info
```

pandas.core.frame.DataFrame.info

```
def info(verbose: bool | None=None, buf: WriteBuffer[str] | None=None, max_cols: int | None=None, memory_usage: bool | str | None=None, show_counts: bool | None=None) -> None
```

Print a concise summary of a DataFrame.

This method prints information about a DataFrame including the index dtype and columns, non-null values and memory usage.

Parameters

DATA Types

```
df.dtypes
```

0	
State	object
City	object
Order Date	datetime64[ns]
Restaurant Name	object
Location	object
Category	object
Dish Name	object
Price (INR)	float64
Rating	float64
Rating Count	int64

dtype: object

df.describe()

	Order Date	Price (INR)	Rating	Rating Count
count	197430	197430.000000	197430.000000	197430.000000
mean	2025-05-01 19:41:20.996808960	268.512920	4.341582	28.321805
min	2025-01-01 00:00:00	0.950000	1.500000	0.000000
25%	2025-03-01 00:00:00	139.000000	4.300000	0.000000
50%	2025-05-02 00:00:00	229.000000	4.400000	2.000000
75%	2025-07-01 00:00:00	329.000000	4.500000	15.000000
max	2025-08-31 00:00:00	8000.000000	5.000000	999.000000
std	NaN	219.338363	0.422585	87.542593

▼ KPI

Total Sales

```
total_sales = df["Price (INR)"].sum()
print("Total sales (INR):",total_sales)
```

Total sales (INR): 53012505.77000001

Average Rating

```
average_rating = df["Rating"].mean()
print("Average Rating:",round(average_rating,2))
```

Average Rating: 4.34

Average Order Value

```
avg_order_value = df["Price (INR)"].mean()
print("Avg Order Value (INR):", round(avg_order_value,2))
```

Avg Order Value (INR): 268.51

Rating Count

```
rating_count = df["Rating Count"].sum()
print("ratings count:", round(rating_count,2))
```

```
ratings count: 5591574
```

Total Orders

```
total_orders = df["Order Date"].count()
print("total_orders:", round(total_orders,2))
```

```
total_orders: 197430
```

CHARTS DESIGN

```
from google.colab import files
uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving swiggy_data.xlsx to swiggy_data (1).xlsx

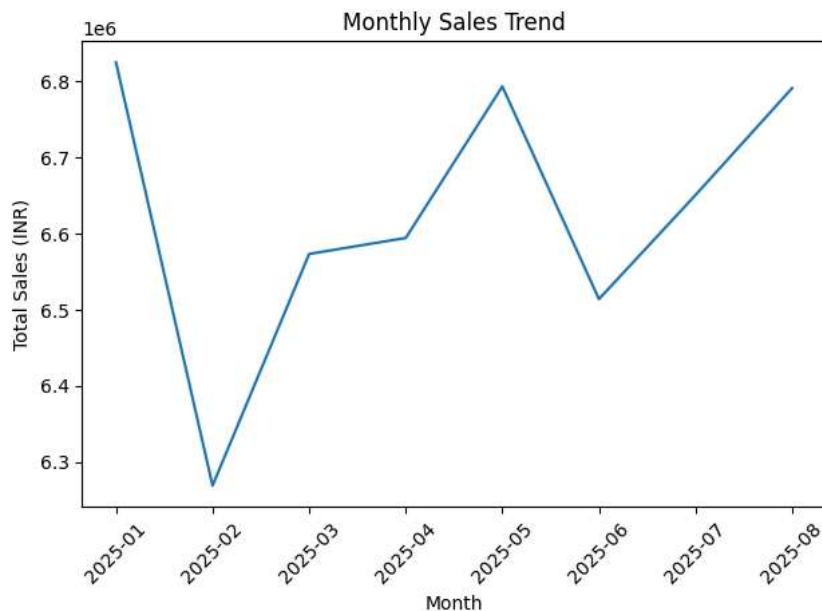
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_excel("swiggy_data.xlsx")

df["Order Date"] = pd.to_datetime(df["Order Date"])
df["YearMonth"] = df["Order Date"].dt.to_period("M").astype(str)

monthly_revenue = df.groupby("YearMonth")["Price (INR)"].sum().reset_index()

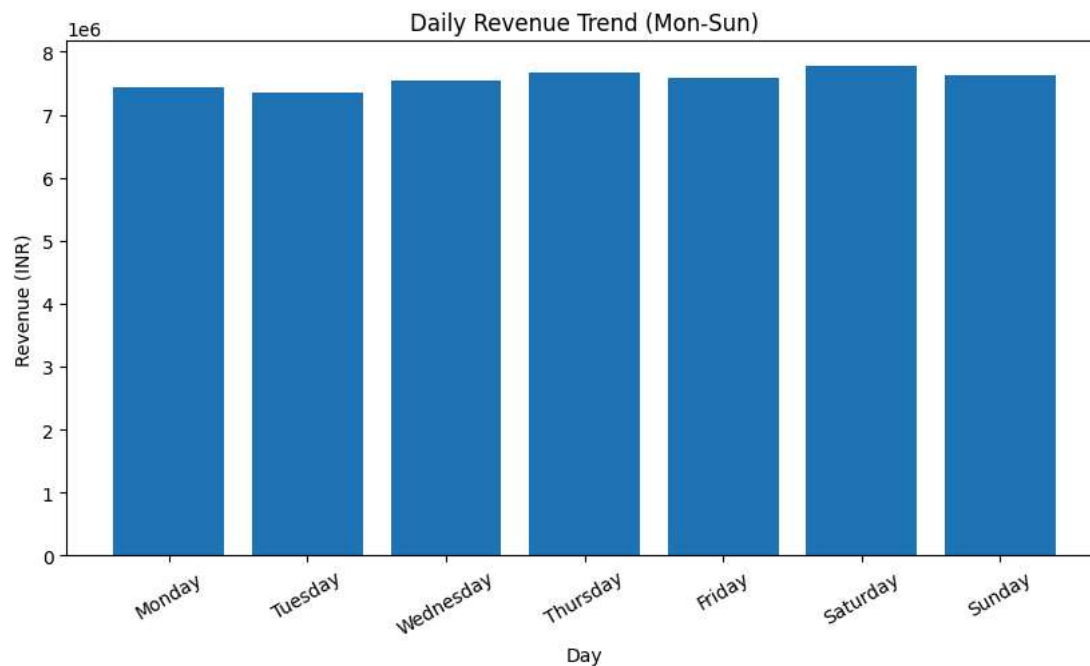
plt.figure()
plt.plot(monthly_revenue["YearMonth"], monthly_revenue["Price (INR)"])
plt.xticks(rotation=45)
plt.xlabel("Month")
plt.ylabel("Total Sales (INR)")
plt.title("Monthly Sales Trend")
plt.tight_layout()
plt.show()
```



```
df["DayName"] = pd.to_datetime(df["Order Date"]).dt.day_name()

daily_revenue = (
    df.groupby("DayName")["Price (INR)"].sum().reindex(["Monday", "Tuesday", "Wednesday", "Thursday", '])
```

```
)  
plt.figure(figsize=(10,5))  
plt.bar(daily_revenue.index,daily_revenue.values)  
plt.title("Daily Revenue Trend (Mon-Sun)")  
plt.xlabel("Day")  
plt.ylabel("Revenue (INR)")  
plt.xticks(rotation=30)  
  
plt.show()
```



```
non_veg_keywords = [  
    "chicken", "egg", "fish", "mutton",  
    "prawn", "biryani", "kabab", "kebab",  
    "non-veg", "non-veg"  
]  
df["Food Category"] = np.where(  
    df["Dish Name"].str.lower().str.contains("/".join(non_veg_keywords), na = False),  
    "Non-Veg",  
    "Veg"  
)
```

```
import numpy as np

non_veg_keywords = [
    "chicken", "egg", "fish", "mutton",
    "prawn", "biryani", "kabab", "kebab",
    "non-veg"
]

df["Food Category"] = np.where(
    df["Dish Name"].str.lower().str.contains("|".join(non_veg_keywords), na=False),
    "Non-Veg",
    "Veg"
)

food_revenue = (
    df.groupby("Food Category")["Price (INR)"]
    .sum()
    .reset_index()
)
```

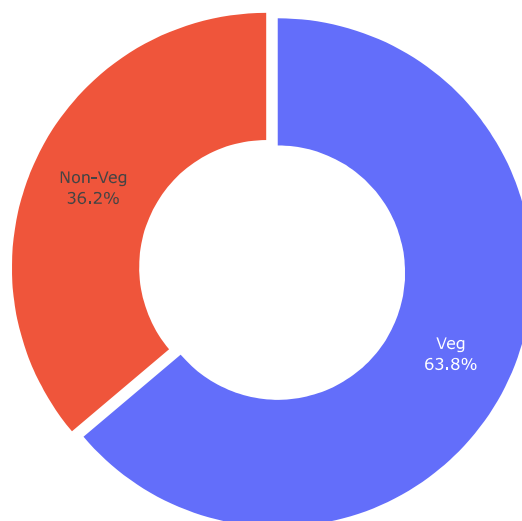
```
fig = px.pie(
    food_revenue,
    values="Price (INR)",
    names="Food Category",
    hole=0.5,
    title="Revenue Contribution: Veg vs Non-Veg",
)

fig.update_traces(
    textinfo="percent+label",
    pull=[0.05, 0]
)

fig.update_layout(
    height = 500,
    margin=dict(t=60,b=40,l=40,r=40)
)

fig.show()
```

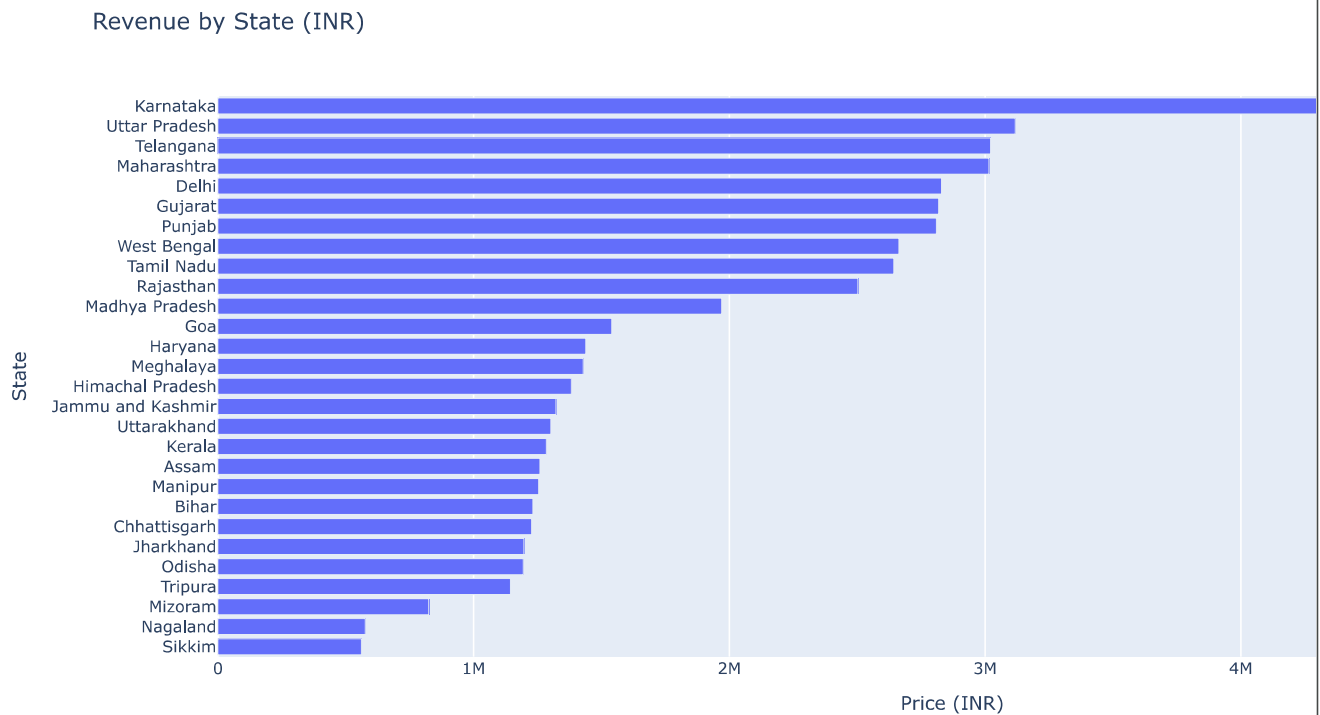
Revenue Contribution: Veg vs Non-Veg



▼ Total Sales by State

```
fig = px.bar(
    df.groupby("State", as_index=False)["Price (INR)"].sum()
    .sort_values("Price (INR)", ascending=False),
    x="Price (INR)",
    y="State",
    orientation="h",
    title="Revenue by State (INR)"
)

fig.update_layout(height=600, yaxis=dict(autorange="reversed"))
fig.show()
```



▼ Top 5 Cities by Sales

```
top_5_cities = df.groupby("City")["Price (INR)"] \
    .sum() \
    .nlargest(5) \
    .sort_values() \
    .reset_index()

fig = px.bar(
    top_5_cities,
    x="Price (INR)",
    y="City",
    orientation="h",
    title="Top 5 cities by sales (INR)",
    color_discrete_sequence=["red"]
)

fig.show()
```

